KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

REACT JS PROJECT REPORT ON

Event Discussion Platform

Submitted by

POORNACHANDRIKA.S

24BIT135

Under the Guidance of

Mr.ARAVIND

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

DECLARATION

I hereby declare that the project entitled "**Event Discussion Platform"** submitted in partial fulfillment of the requirements for the course [**REACT.JS]** is a **record of my original work** carried out under the guidance of **[Mr.ARAVIND]** Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____POORNACHANDRIKA.S_____

CERTIFICATE

This is to certify that the project entitled **“EVENT DISCUSSION PLATFORM”** is a **bona fide work** carried out by **[POORNACHANDRIKA.S] [ll -IT-”B”]**, [**INFORMATION TECHNOLOGY]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[REACT JS]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place:coimbatore

Date:09.04.2026

Submitted for the Viva-Voce Examination held on _____9.04.2026_____

Guide:

(Technical Trainer, KGCAS).

_____

Head of the Department:

(Technical Head, KGCAS)

_____

Table of Contents

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Background of the Study	
1.2	Problem Statement	
1.3	Objectives of the Project	
1.4	Purpose and Significance	
1.5	Scope and Limitations	
1.6	Methodology Overview	
2	**System Analysis**	
2.1	Existing System	
2.2	Drawbacks of the Existing System	
2.3	Proposed System	
2.4	Features of the Proposed System	
2.5	Feasibility Study	
2.6	Requirement Analysis	

3	**System Design**	
3.1	System Architecture	
3.2	Use Case Diagram	
3.3	Data Flow Diagram (DFD)	
3.4	Entity Relationship Diagram (ERD)	
3.5	User Interface Design	
4	**System Implementation**	
4.1	Development Tools	
4.2	Module Description	
4.3	Algorithms	
4.4	System Integration	
4.5	Security and Validation	
5	**System Testing**	
5.1	Testing Methodology	
5.2	Test Cases	
5.3	Error Handling	

5.4	Performance	
6	**Input and Output Screens**	
6.1	System Outputs	
6.2	Discussion	
6.3	Comparison	
6.4	User Feedback	
7	**Conclusion and Future Enhancement**	
7.1	Summary	
7.2	Limitations	
7.3	Future Scope	
8	**References**	

Appendices

- Source Code Snippets
- Screenshots of Web Pages

ABSTRACT

The Event Discussion Platform is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The main aim of this project is to provide a centralized platform where users can explore events, book tickets, and actively participate in discussions related to those events.

Unlike traditional event management systems that focus only on displaying event details, this platform emphasizes user interaction and engagement through a structured discussion system.
The application allows administrators to create, update, and delete events by providing details such as title, description, date, and venue. Users can view these events and book them by entering basic details like name, email, and phone number.

One of the key features of the system is the discussion module, where users can post comments, reply to others, like comments, and engage in meaningful conversations. The nested (tree-structured) comment system helps maintain organized and easy-to-follow discussions.

Additionally, the platform includes a basic notification mechanism that alerts users when they are tagged in a comment or when someone replies to their comment. This enhances user engagement and keeps users informed about interactions. The system also differentiates between admin and user roles to ensure proper access control and functionality.

In this system, the admin can create, update, and delete events by providing details such as title, description, date, and venue. Users can explore these events and book them by entering basic information like name, email, and phone number. The platform also includes a discussion feature where users can post comments, reply to others, and interact easily.

Overall, this project provides an interactive and efficient solution for event management combined with real-time user communication. It demonstrates the use of modern web technologies to build a responsive, scalable, and user-friendly application.

Chapter 1: INTRODUCTION

1.1 Background of the Study :

With the rise of digital platforms, communication and discussion about events have shifted online. However, many platforms lack structured discussion features focused on specific events. This project aims to provide a dedicated platform for event-based discussions.

In many existing systems, users can view events but cannot actively interact or share their opinions in an organized way. This creates a gap between event information and user engagement. To overcome this issue, the proposed system combines event management with a discussion feature, allowing users to comment, reply, and interact with others.

The platform also supports event booking and basic user interaction, making it more useful and interactive. By integrating these features, the system provides a simple and effective solution for both event management and communication.

1.2 Problem Statement

Existing platforms like Facebook and Twitter combine all types of content in a single space, which often makes it difficult for users to focus on specific events or meaningful discussions. Important event-related conversations get lost among unrelated posts, reducing clarity and engagement. Therefore, there is a clear need for a dedicated platform that allows users to participate in structured, organized discussions centered around specific events, enabling better interaction, information sharing, and user experience.

1.3 Objectives of the Project

The main objective of this project is to create a simple website where users can view events, book them, and discuss about them. Admin can add and manage events, while users can comment, reply, and interact with others. The aim is to make event management and discussion easy and user-friendly

Overall, the project aims to provide an interactive platform for event management and user engagement in a simple and efficient way.

1.4 Purpose and Significance of the Study

The purpose of this project is to develop a simple and user-friendly platform where users can view events, book them, and participate in discussions. It aims to provide a space where users can share their opinions, ask questions, and interact with others about specific events.

The system also helps admins to easily manage events by adding, updating, and deleting event details.

Overall, the purpose is to improve user engagement and make event management and communication more effective.

1.5 Scope and Limitations

Scope:

The Event Discussion Platform allows users to view events, book them, and participate in discussions. It supports features like adding events, commenting, replying, and basic notifications. The system can be used for managing small to medium-level events and improving user interaction. It also provides separate roles for admin and users, making event management easier and more organized

Limitations:

The project has some limitations such as basic security (no advanced authentication), simple notification system (not real-time), and limited scalability for large applications. It does not support online payment for bookings and depends on internet connectivity. The system is mainly designed for learning and demonstration purposes.

1.6 Methodology Overview

The project is developed using the MERN stack with a simple approach. The frontend is built using React.js to create user interfaces, and the backend is developed using Node.js and Express.js to handle data and APIs. MongoDB is used to store all the information like users, events, bookings, and comments.

The system is divided into modules such as event management, booking, and discussion. All these modules are connected and tested to make sure the application works properly. Finally, the project is deployed for online access

Chapter 2: System Analysis

2.1 Existing System

In the existing system, most event platforms only allow users to view event details such as title, date, and venue. Some systems provide basic booking features, but they do not support proper interaction between users. There is no structured discussion system where users can share their opinions, ask questions, or reply to others about a specific event.

As a result, user engagement is limited, and communication about events is scattered across different platforms. This creates a gap between event information and user interaction, making the overall experience less effective.

Moreover, existing systems do not provide a combined platform for both event management and user interaction. Users often need to use separate applications or social media platforms to discuss events, which makes the process inconvenient and unorganized. This lack of integration reduces the overall efficiency and user experience, as all features are not available in a single place.

2.2 Drawbacks of the Existing System:

The existing systems have several drawbacks. They mainly focus only on displaying event details and lack interactive features. There is no proper discussion system for users to share opinions or communicate with others about events. User engagement is limited due to the absence of comments, replies, and interaction features.

Additionally, many systems do not provide structured communication, making it difficult to follow conversations. Notification features are also missing or very basic. Overall, the existing systems are less interactive and do not fully support user involvement

Another drawback of the existing system is the lack of personalization and real-time updates. Users are not notified about important activities such as replies or mentions, which reduces their involvement. Also, the systems are not flexible enough to handle growing user needs, making them less effective for interactive and dynamic event environments.

2.3 Proposed System

The proposed system is a simple Event Discussion Platform where admin can manage events and users can view, book, and discuss them. Users can comment, reply, and interact with others about events. The system also includes basic booking and notification features, making event management and communication easy and user-friendly.

2.4 Features of the Proposed System

The system includes various features to make event management and interaction easy. Admin users can create, update, and delete events with details like title, description, date, and venue. Users can view these events and book them by providing their name, email, and phone number. The platform also supports a discussion system where users can add comments, reply to others, like, edit, and delete comments in a structured way. It includes basic notification functionality for replies and tagging, along with user and admin role management. Overall, the system provides a simple and user-friendly interface for both event management and communication.

2.5 Feasibility Study

The proposed system is feasible as it uses simple and widely available technologies like the MERN stack. It can be developed with basic resources and does not require high cost. The system is easy to implement and maintain, making it suitable for small to medium-level applications. It is also user-friendly and can be accessed easily through a web browser, ensuring practical usability

2.6 Requirement Analysis

The system requires both functional and non-functional requirements to work properly. Functionally, it should allow admin users to manage events by adding, updating, and deleting event details. Users should be able to view events, book them by entering basic information, and participate in discussions through comments and replies. The system should also support features like likes and basic notifications.

Non-functional requirements include a simple and user-friendly interface, fast response time, and reliable data storage using MongoDB. The system should be accessible through a web browser and work smoothly for both admin and user roles.

Chapter 3: System Design

3.1 Introduction :

The Event Discussion Platform is a MERN stack web application that allows users to create, view, and participate in event-based discussions. It provides a centralized system where users can interact through comments, replies, and bookings.

The platform uses React for the frontend, Node.js and Express for the backend, and MongoDB for data storage. It ensures a structured and user-friendly way to manage and discuss event.

3.2 System Architecture

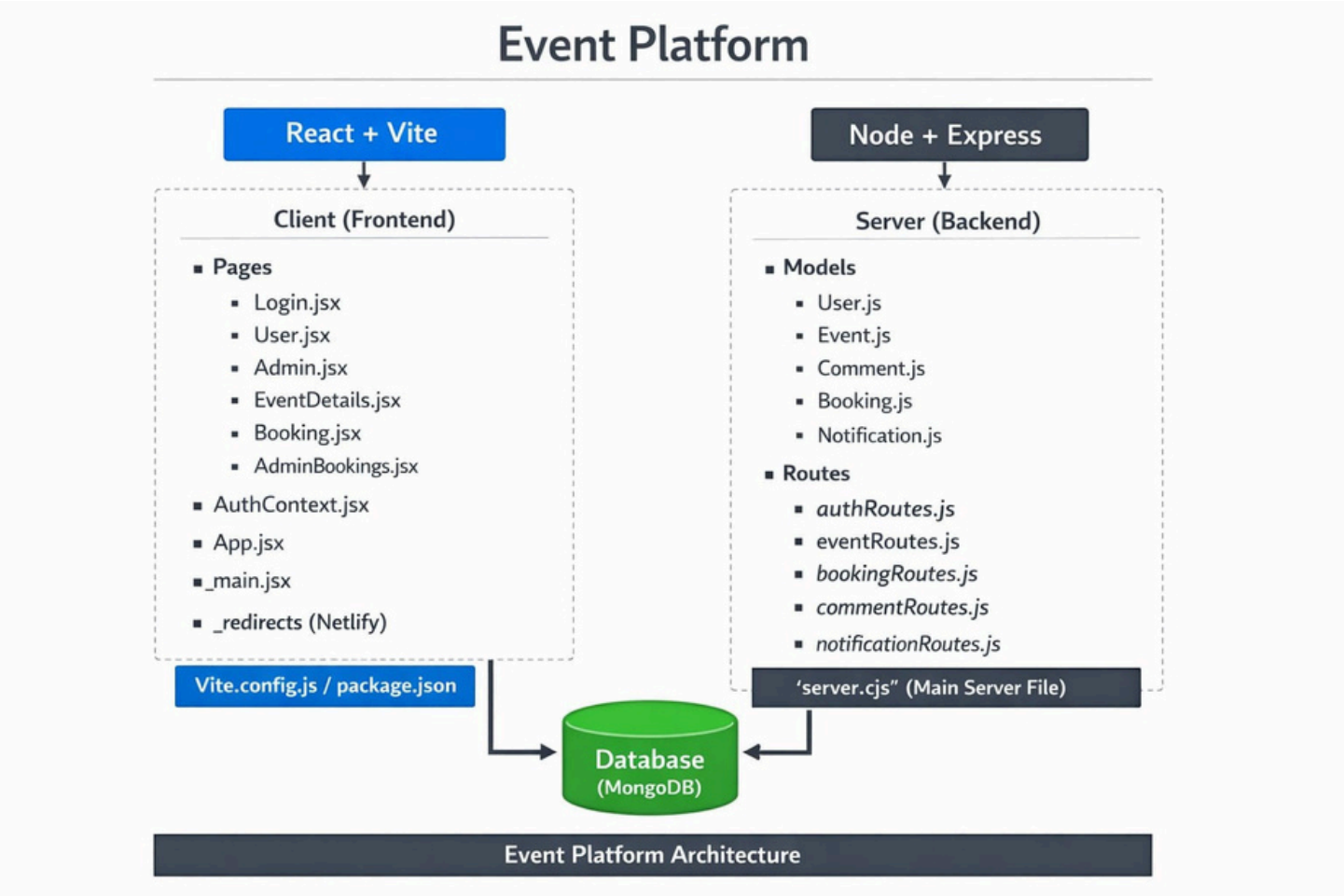
The Event Discussion Platform follows a MERN stack architecture:

Frontend:React.js

Backend: Node.js with Express (server/)

Database: MongoDB

Communication: REST APIs



3.3 Use Case Diagram

1. What is a Use Case Diagram?

A Use Case Diagram is a type of diagram that shows how users interact with a system. It represents different users (called actors) and the actions (called use cases) they can perform in the system.

In this project, the main actors are Admin and User. Admin can add, update, and delete events, while users can view events, book them, and participate in discussions. The diagram helps to understand the system functions clearly and shows the relationship between users and system features.

2. Identify the Main Actors (for your Educational Website)

Actor	**Description**
User(Customer/Attendee)	This is a regular person who uses the platform to browse and book events.
Admin (Event Manager)	Manages the platform by adding/editing/deleting events, viewing bookings, and sending notifications.
System/Backend	Handles authentication, booking processing, and data storage.

3. Identify the Main Use Case

The main use cases in this system are based on the actions performed by Admin and User

For the Admin, the main use cases are creating events, updating event details, and deleting events. These actions help in managing the events effectively.

For the User, the main use cases include viewing events, booking events, and participating in discussions by adding comments, replying, and liking comments

Overall, the key use cases of the system are event management, event booking, and user interaction through discussions.c

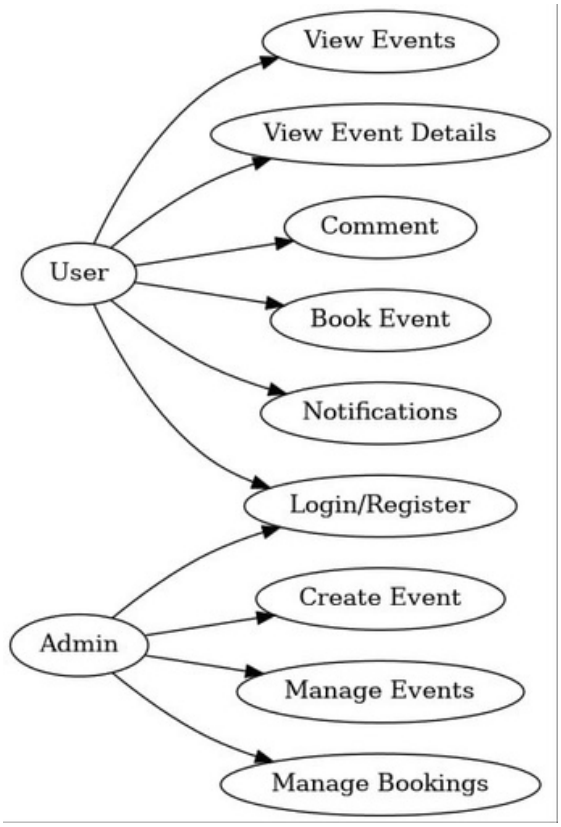
4. Define Relationships

Symbol	Meaning
Solid line	Direct link between actor and use case (e.g., User → Book Event).
<<include>>	Mandatory use case used by another (e.g., Book Event includes Login).
<<extend>>	Optional behavior (e.g., View Event Details extends View Events).

5. Draw the Diagram (Structure Example)

Tools you can use:

- draw.io (free online)
- Lucidchart
- StarUML MS Visio
- AI Tools



6. Example Explanation (Text Description)

Actors:

User – regular attendee.

Admin – platform manager.

Use Cases:

User: Login, View Events, View Event Details, Book Event, Comment.

Admin: Login, Manage Events, View Bookings, Send Notifications.

Relationships:

- *Association:*
- *User → Book Event (direct interaction).*
- *Admin → Manage Events.*
- *Include:*
- *Book Event includes Login (user must login to book).*
- *Extend:*
- *View Event Details extends View Events (optional to see details).*
- *Generalization:*
- *Admin is a User (can do everything a user can plus admin tasks).*

7. Step-by-Step (Using draw.io)

Step 1: Open draw.io

Step 2: Add External Entities

Step 3: Add Main Process

Step 4: Add Data Flows (Arrows)

Step 5: Arrange Properly

Step 6: Final Touch

Admin ----> [Event Discussion Platform] ----> User

Event Details Event InfoUser ----> [System]

Booking / Comments. System ----> User

Confirmation / Discussion

Send Notifications

5. Connect Actors to Use Cases

Draw lines (associations):

User → Login, View Events, View Event Details, Book Event

Admin → Login, Manage Events, View Bookings, Send Notifications

6. Optional Relationships

Include: Book Event includes Login (dashed arrow)

Extend: View Event Details extends View Events (dashed arrow)

Generalization: Admin → User (triangle line)

7. Save

File → Export → PNG/PDF

3.4 Data Flow Diagram (DFD)

1. What is a Data Flow Diagram (DFD)?

A Data Flow Diagram (DFD) is a diagram used to show how data moves through a system. It represents how data is input, processed, stored, and output in the system.

In this project, the DFD shows how users and admin interact with the system, how data like events, bookings, and comments are stored in the database, and how information flows between the frontend, backend, and database. It helps in understanding the overall working of the system clearly.

2. Components

A Data Flow Diagram has four simple components. Process shows the actions in the system like adding events or booking. Data flow shows how data moves between parts of the system. Data store represents where data is saved like the database. External entity represents users like admin and user who interact with the system.

3. Levels of DFD

Level 0 (Context Diagram):

This is the basic level that shows the entire system as a single process with inputs and outputs from users like admin and user.

Level 1:This level breaks the system into main processes such as event management, booking, and discussion.

Level 2:This level gives more detailed explanation of each process, showing how data flows inside each module.

4. Example (Simple Level 0 DFD)

Admin → sends event details → System

User → sends booking details → System

User → sends comments/replies → System

System → displays events → User

System → sends booking confirmation → User

System → shows comments/discussions → User/Admin**:**

2. DFD Symbols You'll Use

Symbol	Meaning
Rectangle	Represents people or systems outside the platform, e.g., User, Admin
Circle / Rounded Rectangle	Represents actions or functions that transform data, e.g., Login, Book Event, Manage Events
Two Parallel Lines	Where data is stored, e.g., User Database, Event Database, Booking Records
Arrow	Shows movement of data between entities, processes, and stores, e.g., User Details, Booking Info

3. Levels of DFD:

Level 0 (Context Diagram): Shows the system as one main process interacting with external entities.

Level 1: Breaks the main process into sub-processes.

Level 2: Shows detailed sub-processes and dataflow.

4. Steps to Create a DFD

Identify the system – Example: Event Platform.

Identify external entities – Example: User, Admin.

Identify processes – Example: Login, Book Event, Manage Events.

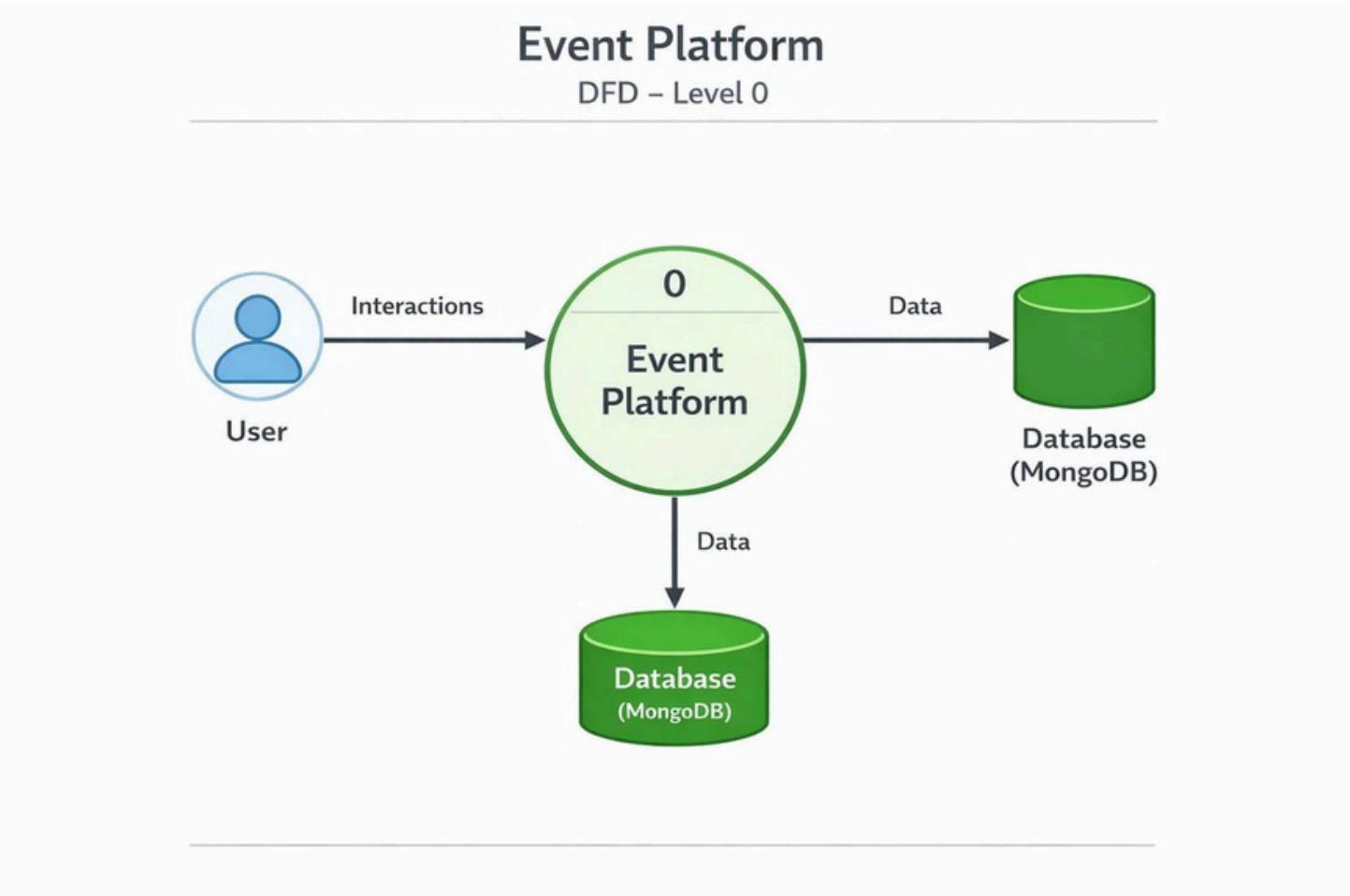
Identify data stores – Example: User Database, Event Database, Booking Records.

Identify data flows – Example: User Details, Event Info, Booking Confirmation.

Draw DFD – Show entities, processes, data stores, and flows.

5. DFD for Educational Website

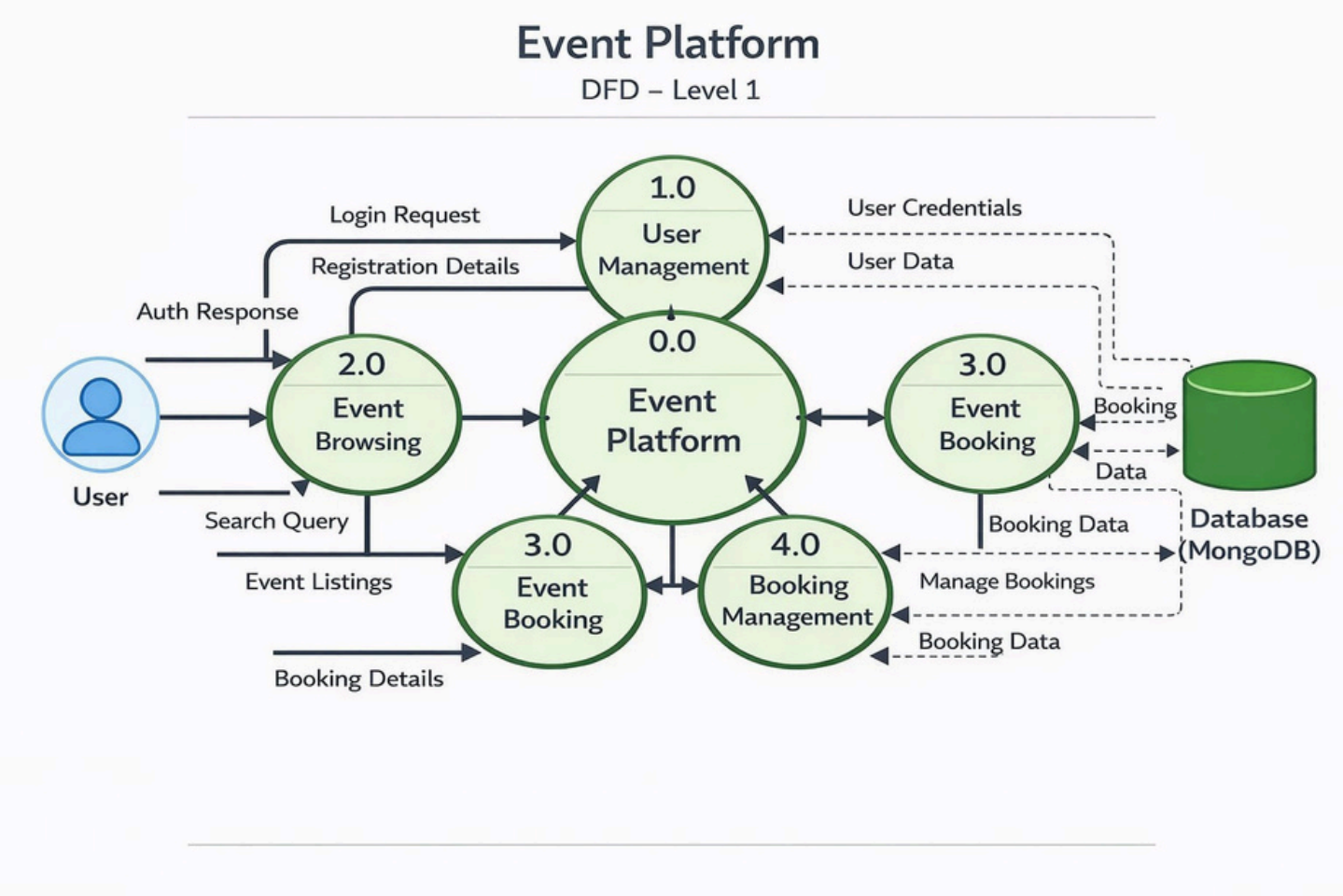
Level 0: Context Diagram



Explanation:

- Student interacts with the system to view courses, read lessons, and take quizzes.
- Admin updates and manages course and quiz content.

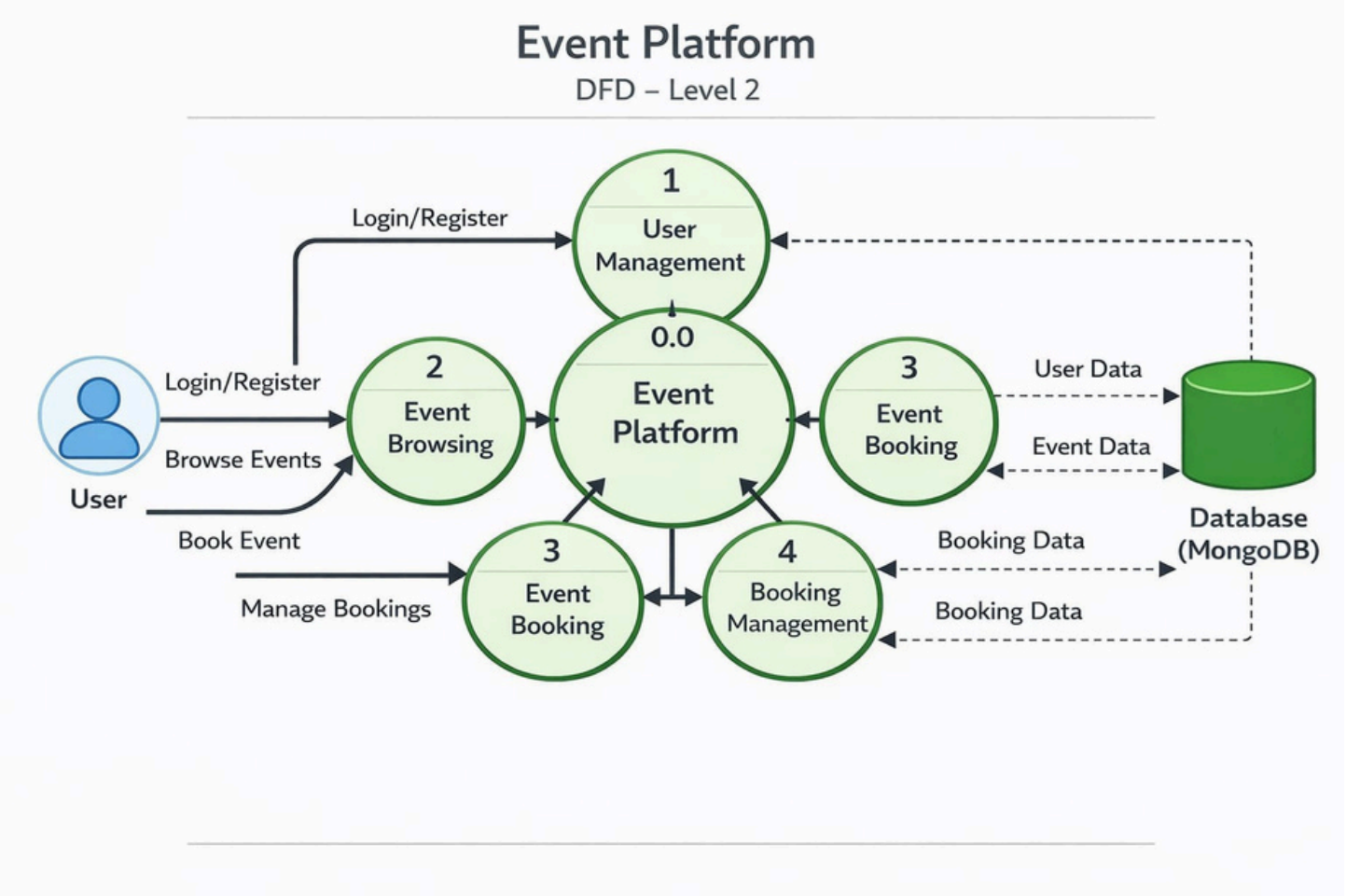
Level 1: Detailed System View



Explanation:

- **Process P1:** Fetches course details from *CourseData Store*.
- **Process P2:** Displays lessons from *Lesson Data Store*.
- **Process P3:** Conducts quizzes using *Quiz Data Store*.
- **Process P4:** Calculates and stores quiz results in *Result Data Store*.

Level 2: Quiz Process (Example Sub-process)



6. Tools to Draw Your DFD

Draw.io (diagrams.net) – Free, easy to use online tool.

Lucidchart – Online diagramming tool, has DFD templates.

Microsoft Visio – Professional software for diagrams (paid).

Creately – Online tool with simple drag-and-drop shapes.

Pencil & Paper – Simple and quick for rough sketches.

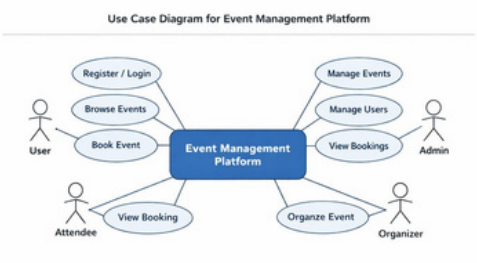
3.5 Entity Relationship Diagram (ERD)

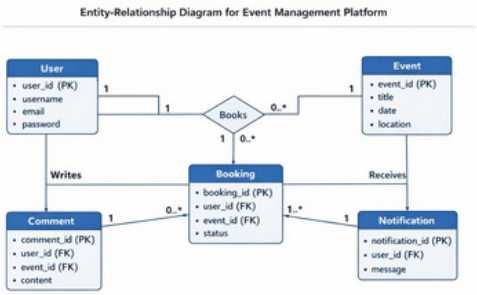
Entities and Relationships:

- **Course** (Course_ID, Title, Description)
- **Lesson** (Lesson_ID, Course_ID, Content)
- **Quiz** (Quiz_ID,Course_ID,Question, Answer, Score)

Relationships:

- A *Course* can have many *Lessons*.
- A *Course* can have multiple *Quizzes*.





3.6 User Interface Design

User Interface (UI) Design – Event Platform

Definition:

UI design is how the platform looks and how users interact with it.

Objectives:

Easy to use

Clear and simple

Consistent design

Works on desktop and mobile

Main Pages:

Login Page

User Page (View Events)

Event Details Page

Booking Page

Admin Page (Manage Events)

Admin Bookings Page

UI Features:

Navigation Menu

Buttons and Forms

Feedback Messages

Responsive Design

Chapter 4: System Implementation

4.1 Introduction

System implementation is the stage where the event platform design is converted into a working web application. This phase includes coding, integrating the frontend and backend, testing functionality, and ensuring that the website performs as expected. The Event Platform is implemented using React (frontend) and Node.js with Express (backend), with MongoDB Atlas as the database for storing event and booking data.

4.2 Development Tools and Technologies

Tool / Technology	Purpose
React.js	Builds interactive and responsive frontend pages.
Node.js & Express	Handles backend server, API routes, and logic.
MongoDB Atlas	Stores users, events, bookings, and comments.
Visual Studio Code	Code editor for development and testing.
Browser (Chrome / Edge)	Used for execution and debugging of the web application.
Netlify / Vercel	Deploys frontend and backend online.

4.3 System Modules Description

Module	Description
login Module	Allows users and admin to log in and authenticate.
User Module	Displays available events and details to the user.
Event Details Module	Shows full information about selected events.
Booking Module	Enables users to book events and view their bookings.
Admin Bookings Module	Lets admin view all bookings and manage them.

All modules are connected through API calls and React state management to ensure smooth navigation.

4.4 Implementation Details

Frontend Design:

The interface is built with React components, styled with CSS for a clean and responsive layout.

Dynamic Behavior:

React state and props are used to dynamically render events, bookings, and notifications.

Event Handling:

Button clicks and form submissions are linked to event handlers (e.g., onClick, onSubmit).

Booking Logic:

When a user books an event, the backend verifies the availability and stores the booking in MongoDB. Confirmation is displayed on the frontend.

Data Management:

All data including users, events, and bookings is handled via MongoDB collections and fetched using Axios API calls.

4.5 Algorithm / Logic of Core Function

Algorithm for Booking an Event:

User selects an event.

Check if slots are available.

If available, save booking in the database.

Display confirmation to the user.

Algorithm for Displaying Event List:

Fetch all events from the backend.

Loop through the events.

Render event cards dynamically using React components.

Display details when a user clicks on an event.

4.6 Security and Validation

Input validation is implemented on forms to prevent empty submissions.

Only registered users can book events.

Backend validates all requests before saving data to the database.

Error messages are displayed for invalid inputs or failed operation.

4.7 Integration

All modules are integrated using React routing and API calls. Users can navigate between the homepage, event details, and booking pages without reloading. Admin functions are also integrated to manage events and bookings seamlessly.

4.8 Output Screens (Snapshots)

Include labeled screenshots for:

Homepage: List of available events.

Event Details Page: Event information and description.

Booking Page: Form for booking an event.

Admin Page: Add/Edit/Delete events.

Admin Bookings Page: View and manage all bookings.

Chapter 5: System Testing

5.1 Introduction

System testing ensures that the Event Platform works as intended. It checks that all features function correctly and that users can interact with the system smoothly. Testing was performed for login, event viewing, booking, and admin management to verify proper functionality.

5.2 Objectives of Testing

Ensure all modules (Login, User, Event Details, Booking, Admin) work correctly.

Identify and fix any errors or bugs.

Confirm that user inputs give correct outputs.

Verify smooth operation across different browsers and devices.

5.3 Types of Testing Performed

Unit Testing

Each React component and backend API route was tested separately.

Example: Login component, Booking function, Event API.

b) Integration Testing

After unit testing, modules were combined.

Example: Navigating from Homepage → Event Details → Booking.

c) System Testing

Full platform was tested to ensure all functional requirements are met.

Checked features like booking events, viewing details, and admin management.

d) User Acceptance Testing (UAT)

Sample users tested the platform.

Feedback was collected on usability, layout, and booking flow.

e) Cross-Browser and Device Testing

Tested on Chrome, Firefox, Edge, and mobile devices to ensure responsive design.

5.4 Test Plan

Test ID	Module	Test Description
T01	login	User/Admin login
T02	User Page	Display events list
T03	Event Details	Show event info
T04	Booking	Book selected event
T05	Admin	Add/Edit/Delete events
T06	Admin Bookings	View all bookings
T07	Navigation	Move between modules
T08	Responsiveness	Mobile view UI Layout adjusts properly Pass

5.5 Test Data and Execution

Sample data were created for users, events, and bookings.

API calls and React components were tested to ensure correct data retrieval and display.

Outputs were checked against expected results.

5.6 Error Handling

Input validation prevents empty or invalid bookings.

Backend checks ensure data integrity before saving.

Error messages guide users if an action fails.

5.7 Performance and Usability Testing

Performance: Pages load quickly (<2 seconds).

Usability: Users found navigation simple and intuitive.

Accessibility: Supports mobile devices and readable fonts.

5.8 Results and Observations

All modules worked correctly.

The platform performed well on different devices and browsers.

Users confirmed the system is easy to use, interactive, and efficient.

No major defects were found in the final testing phase.

Chapter 6: Results and Discussion

6.1 Introduction

This chapter presents the results obtained after implementing and testing the Event Platform. It shows how each module performed and how well the system met its objectives, such as event viewing, booking, and admin management.

6.2 System Outputs

During execution, the platform produced the following results:

Homepage: Displayed a list of upcoming events with titles and short descriptions.

Event Details Page: Showed full information about selected events dynamically.

Booking Page: Allowed users to book events and confirmed the booking instantly.

Admin Page: Enabled the admin to add, edit, or delete events.

Admin Bookings Page: Showed all bookings with options to manage them.

All pages were responsive and accessible on desktops and mobile devices.

6.3 Performance Analysis

Responsiveness: Pages loaded quickly (< 2 seconds).

Accuracy: Booking and event management functions worked correctly.

Stability: No runtime errors occurred during testing.

Cross-Browser Compatibility: Verified on Chrome, Edge, and Firefox, with consistent performance.

6.4 User Feedback

A group of test users evaluated the platform. Feedback included:

Navigation was simple and intuitive.

Layout was clear and readable.

Users suggested adding notifications, search, and user profile features in future updates.

Chapter 7: Conclusion and Future Enhancement

7.1 Conclusion

The Event Platform project successfully demonstrates how modern web technologies can be used to create an interactive and user-friendly event management system. By utilizing React for frontend, Node.js/Express for backend, and MongoDB for data storage, the platform allows users to view events, check details, book events, and for admins to manage events and bookings.

The project achieved its main objectives of providing a simple, responsive, and functional platform. Testing confirmed that all modules—login, user event viewing, booking, and admin management—operate correctly and reliably. The system meets functional and non-functional requirements such as usability, responsiveness, and stability.

This project provides a solid foundation for building a more advanced event management system with additional features in the future.

7.2 Limitations

While the platform meets core requirements, the current version has some limitations:

No advanced user roles beyond basic admin and user.

Limited notification or reminder system for bookings.

No payment integration for paid events.

Basic reporting and analytics for admin.

7.3 Future Enhancement

The platform can be improved and scaled in several ways:

User Accounts: Implement secure login and registration with roles.

Payment Integration: Enable online payment for events.

Advanced Admin Panel: Add features to manage events, bookings, and reports more efficiently.

Notifications: Send email/SMS reminders for booked events.

Analytics Dashboard: Provide event statistics, booking trends, and user activity.

Mobile App: Develop a mobile-friendly version or app for easier access.

Cloud Deployment: Host on cloud platforms like AWS, Firebase, or Vercel for real-time updates and multi-user access.

Chapter 8: References

8.1 Books & Journals

Web Technologies: HTML, CSS, JavaScript, Node.js, Express, MongoDB — Uttam K. Roy, Oxford University Press.

Learning React: Functional Web Development with React and Redux — Alex Banks & Eve Porcello, O'Reilly Media.

Modern Web Development — Peter Gasston, Wiley.

8.2 Online Resources

MDN Web Docs — https://developer.mozilla.org

W3Schools — https://www.w3schools.com

React Official Documentation — https://reactjs.org

Node.js Official Documentation — https://nodejs.org

MongoDB Documentation — https://www.mongodb.com/docs

GeeksforGeeks Web Development Articles — https://www.geeksforgeeks.org

8.3 Tools & Frameworks Used

Editor: Visual Studio Code

Languages: HTML5, CSS3, JavaScript (ES6), Node.js

Frontend Framework: React.js, Bootstrap 5 (for responsive UI)

Backend: Node.js + Express

Database: MongoDB Atlas (cloud database)

Browser: Google Chrome / Microsoft Edge

Appendices

Appendix A – Source Code

Include the main source files for the Event Platform:

App.jsx / index.jsx – Main React application structure and routing.

Login.jsx – Login page for users and admin authentication.

User.jsx – Displays list of available events for users.

EventDetails.jsx – Shows detailed information about a selected event.

Booking.jsx – Handles booking functionality and confirmation.

Admin.jsx – Admin panel to add, edit, or delete events.

AdminBookings.jsx – View and manage all bookings as admin.

AuthContext.jsx – Handles authentication and user session state.

style.css / index.css – Styling and responsive layout for all pages.

(Attach essential code snippets here — show only key functions or components to keep it readable.)

Appendix B – Screenshots of Web Pages

Figure 1.1: Homepage displaying upcoming events.

Figure 1.2: Event Details Page showing event description and timings.

Figure 1.3: Booking Page with form to confirm event registration.

Figure 1.4: Admin Page showing event management options.

Figure 1.5: Admin Bookings Page displaying all user bookings.

Figure 1.6: Responsive mobile view of homepage and booking page.

Appendix C – User Manual / Installation Guide

Steps to Run the Project:

Download all project files (client and server folders).

Open terminal in the server folder and run npm install to install dependencies.

Start the backend server using node server.cjs.

Open terminal in the client folder, run npm install and then npm run dev to start the frontend.

Open the browser at the local URL shown (e.g., http://localhost:5173).

Navigate through the platform: browse events, view details, book events, or manage events as admin.

System Requirements:

OS: Windows / macOS / Linux

Browser: Latest Chrome, Firefox, or Edge

RAM: Minimum 2 GB

Storage: 500 MB free (for project files and dependencies)